



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Pardilla, Dianna Francesca M.
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects

- If-Else statements
 - Loops (for loop)
 - Functions / methods
2. Understand the environment:
- Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... × + ▾
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |
```

Your Code - Laboratory Tasks

```
from dataclasses import dataclass
from enum import Enum
import random
import time

# CLI THEME PALETTE & VISUAL ASSETS
class TerminalPalette:
    RESET    = "\033[0m"
    BOLD     = "\033[1m"
    DIM      = "\033[2m"
    CYAN     = "\033[96m"
    GREEN    = "\033[92m"
    YELLOW   = "\033[93m"
    MAGENTA  = "\033[95m"
    RED      = "\033[91m"
    BLUE     = "\033[94m"

class RoomStatus(str, Enum):
    DIRTY = "Dirty"
    CLEAN = "Clean"

# TASK 1: ENVIRONMENT SETUP & ABSTRACTION
class DualChamberEnvironment:
    """Manages the two-room spatial state dictionary and mutation
    hooks."""

    def __init__(self, room_a_state: str = "Clean", room_b_state: str =
"Clean"):
        # Task 1: Initialize two rooms using a Python dictionary
        self._chambers: dict[str, RoomStatus] = {
            'A': RoomStatus.DIRTY if room_a_state.strip().capitalize() ==
"Dirty" else RoomStatus.CLEAN,
            'B': RoomStatus.DIRTY if room_b_state.strip().capitalize() ==
"Dirty" else RoomStatus.CLEAN
        }
```

```

@property
def chambers(self) -> dict[str, str]:
    return {room: status.value for room, status in
self._chambers.items()}

def is_dirty(self, room: str) -> bool:
    """Task 1: Sensor function to verify chamber dirt presence."""
    return self._chambers.get(room) == RoomStatus.DIRTY

def clean_room(self, room: str) -> None:
    """Task 1: Actuator hook to sanitize the designated room."""
    if room in self._chambers:
        self._chambers[room] = RoomStatus.CLEAN

# TASK 2: RULE-BASED REFLEX AGENT
class ReflexVacuumAgent:
    """Production-rule reflex agent executing condition-action
directives."""

    def __init__(self, origin: str = 'A'):
        self.location: str = origin.upper()

    def move(self) -> None:
        """Task 2: Navigates across the binary topological room layout."""
        self.location = 'B' if self.location == 'A' else 'A'

    def perceive_and_act(self, env: DualChamberEnvironment) -> tuple[str,
str]:
        """
        Task 2: Evaluate sensory percept and trigger conditional action.
        - Rule 1: If current chamber is Dirty -> Purge contaminant.
          - Rule 2: If current chamber is Clean -> Relocate to
counter-chamber.
        """
        origin = self.location

        # Rule 1

```

```

        if env.is_dirty(origin):
            env.clean_room(origin)
            return ("RULE_1", f"Sanitized dirty chamber [{origin}]")

    # Rule 2
    self.move()
    return ("RULE_2", f"Relocated chamber: [{origin}] -> [{self.location}]")

# TASK 3: SIMULATION RUNNER & HUD PRINTER
def print_telemetry_card(step_num: int, origin_loc: str, rule_fired: str,
action_desc: str, room_states: dict[str, str]) -> None:
    """Formats and prints an ASCII telemetry HUD card for each execution
    step."""
    p = TerminalPalette

    # State color tags
    def badge(status: str) -> str:
        return f"{p.RED}DIRTY{p.RESET}" if status == "Dirty" else
f"{p.GREEN}CLEAN{p.RESET}"

    border =
f"{p.DIM}+-----+
{p.RESET}"

    rule_tag = f"{p.YELLOW}[CLEAN]{p.RESET}" if rule_fired == "RULE_1"
else f"{p.CYAN}[RELOCATE]{p.RESET}"

    print(border)
    print(f"| {p.BOLD}CYCLE {step_num:02d}{p.RESET} :: {rule_tag}
{action_desc:<37} |")
    print(f"| {p.DIM}Percept Position :{p.RESET} Chamber
[{p.MAGENTA}{origin_loc}{p.RESET}] |")
    print(f"| {p.DIM}Chamber Telemetry:{p.RESET} Room A ->
{badge(room_states['A']):<18} Room B -> {badge(room_states['B']):<18} |")
    print(border)

```



```

        prior_loc = agent.location
        rule_id, telemetry = agent.perceive_and_act(env)
        print_telemetry_card(step, prior_loc, rule_id, telemetry,
env.chambers)

    # Interactive loop control
    retry = input(f"\n{p.MAGENTA}» Rerun 2-Chamber simulation? (y/N):
{p.RESET}").strip().lower()
    if retry != 'y':
        break

# BONUS TASK: 3-CHAMBER VISUAL MATRIX & GRID ENGINE
class MultiChamberEnvironment:
    """Dynamic spatial matrix with variable capacity (Rooms A, B, and
C)."""

    def __init__(self, states: dict[str, str]):
        self.rooms = {k: RoomStatus.DIRTY if v.strip().capitalize() ==
"Dirty" else RoomStatus.CLEAN for k, v in states.items()}

    def is_dirty(self, room: str) -> bool:
        return self.rooms.get(room) == RoomStatus.DIRTY

    def clean(self, room: str) -> None:
        self.rooms[room] = RoomStatus.CLEAN

    def dirty_nodes(self) -> list[str]:
        return [node for node, state in self.rooms.items() if state ==
RoomStatus.DIRTY]

class StochasticMatrixAgent:
    """Agent that handles dynamic chambers and targets dirty rooms
randomly."""

    def __init__(self, initial_node: str = 'A'):
        self.node = initial_node.upper()

```

```

def cycle(self, env: MultiChamberEnvironment) -> str:
    curr = self.node
    if env.is_dirty(curr):
        env.clean(curr)
        return f"Purged contaminant at Room {curr}"

    dirty_set = [r for r in env.dirty_nodes() if r != curr]
    if dirty_set:
        self.node = random.choice(dirty_set)
        return f"Vector Lock: Shifted to Dirty Node [{self.node}]"

    adjacent = [r for r in env.rooms.keys() if r != curr]
    self.node = random.choice(adjacent)
    return f"Patrol Vector: Sweeping Clean Node [{self.node}]"

def render_isometric_grid(room_dict: dict[str, RoomStatus], current_loc: str) -> None:
    """Draws a clean, styled spatial grid showing the agent's live coordinates."""
    p = TerminalPalette

    def render_pod(chamber_id: str) -> tuple[str, str]:
        is_here = (current_loc == chamber_id)
        is_polluted = (room_dict[chamber_id] == RoomStatus.DIRTY)

        status_txt = f"{p.RED}DIRTY{p.RESET}" if is_polluted else f"{p.GREEN}CLEAN{p.RESET}"
        cursor_txt = f"{p.YELLOW}[AGENT]{p.RESET}" if is_here else ""

        line_status = f"| Status : {status_txt:<15} "
        line_cursor = f"| Pos      : {cursor_txt:<18} "
        return line_status, line_cursor

```

```
                                box_top =
"+-----+-----+-----+"
-----+"
    box_header = f"| Chamber A           | Chamber B           | 
Chamber C           | "
                                box_div =
"+-----+-----+-----+"
-----+"
                                box_bottom =
"+-----+-----+-----+"
-----+"


s_a, c_a = render_pod('A')
s_b, c_b = render_pod('B')
s_c, c_c = render_pod('C')


print(f"{p.DIM}{box_top}{p.RESET}")
print(f"{p.BOLD}{box_header}{p.RESET}")
print(f"{p.DIM}{box_div}{p.RESET}")
print(f"{s_a}{s_b}{s_c}|"")
print(f"{c_a}{c_b}{c_c}|"")
print(f"{p.DIM}{box_bottom}{p.RESET}")


def run_bonus_simulation() -> None:
    p = TerminalPalette
    while True:

print(f"\n{p.MAGENTA}|_____|
        {p.RESET}")
            print(f"{p.MAGENTA} |{p.RESET}                {p.BOLD}BONUS: 3-CHAMBER
VISUAL MATRIX SIMULATION{p.RESET}                {p.MAGENTA} |{p.RESET}")

print(f"{p.MAGENTA}|_____|
        {p.RESET}\n")

states = {
```

```

        'A': input(f"{p.DIM}» Chamber A State (Clean/Dirty):
{p.RESET}"),
        'B': input(f"{p.DIM}» Chamber B State (Clean/Dirty):
{p.RESET}"),
        'C': input(f"{p.DIM}» Chamber C State (Clean/Dirty):
{p.RESET}")
    }

    start = input(f"{p.DIM}» Deploy Agent to Chamber (A/B/C):
{p.RESET}").strip().upper()
    start_node = start if start in ['A', 'B', 'C'] else 'A'

    try:
        cycles = int(input(f"{p.DIM}» Step Limit: {p.RESET}"))
    except ValueError:
        cycles = 4

    env = MultiChamberEnvironment(states)
    agent = StochasticMatrixAgent(start_node)

    print(f"\n{p.BOLD}T-0 MATRIX INITIALIZATION:{p.RESET}")
    render_isometric_grid(env.rooms, agent.node)

    for c in range(1, cycles + 1):
        act = agent.cycle(env)
        print(f"\n{p.CYAN}STEP {c:02d} ACTUATOR:{p.RESET} {act}")
        render_isometric_grid(env.rooms, agent.node)

    retry = input(f"\n{p.MAGENTA}» Rerun 3-Chamber simulation? (y/N):
{p.RESET}").strip().lower()
    if retry != 'y':
        break

# MAIN COMMAND INTERFACE
def main():
    p = TerminalPalette
    while True:

```


Answer the Questions ↓

1. It literally only lives by two simple brain-dead vibes:
 - a. **Rule 1 (The Fixer):** If the room it's currently sitting in is Dirty, clean it up and stay put for that turn.
 - b. **Rule 2 (The Wanderer):** If the room is already Clean, pack up and switch to the other room ($A > B$ or $B > A$).
2. The decision is purely reactive based on sensory input from its current chamber via `env.is_dirty(self.location)`:
 - a. **When to clean:** Cleaning is top priority. The second it senses dirt (`is_dirty()` is `True`), it immediately hits clean and ends the turn right there. It doesn't move.
 - b. **When to move:** If it checks the floor and sees it's already spotless (`is_dirty()` is `False`), it goes "my job here is done" and flips over to the opposite room.
3. It gets stuck in an endless loop of pacing back and forth:
 - a. Because neither room is ever dirty, Rule 1 never triggers.
 - b. So it just runs Rule 2 non-stop: $A > B > A > B > A \dots$ pacing like it forgot its keys until the step counter finally runs out and shuts it off. Zero cleaning happens.
4. Right now, the code uses a hardcoded binary switch ('B' if `loc == 'A'` else 'A'), which breaks the second you throw in a Room C. To scale it up:
 - a. Put the rooms on a track (Round Robin): Put them in a list like ['A', 'B', 'C', 'D'] and just make it loop around in order using modulo math (`next_index = (current + 1) % total_rooms`).
 - b. Target the mess directly (like in the bonus 3-chamber code): Give it a scanner so if its current room is clean, it grabs a list of actual dirty rooms and hops straight over to one instead of guessing.
 - c. Give it an off switch or idle mode: If all rooms are verified clean, let the poor thing sleep or go on standby instead of pacing between doors forever!